Appendix B. Recap of streams in Java 8

This appendix is a refresher on Java 8 streams and the aspects of basic functional programming that are associated with them. If you are not familiar with the basic syntax of Java 8 lambda expressions, or the philosophy that underlies their design, you should read a basic text first to familiarize yourself with those concepts, such as ***Modern Java in Action: Lambdas, Streams, Functional and Reactive Programming, 2nd ed.***, by Raoul-Gabriel Urma, Mario Fusco, and Alan Mycroft (Manning, 2018). Java 8 introduced lambda expressions as part of *Project Lambda*, the overall goals of which can be summarized as follows:

- Allow developers to write cleaner and more concise code.
- Provide a modern upgrade to the Java Collections libraries.
- Introduce an abstraction that allows for convenient use of basic functional idioms.

In this appendix, we discuss the upgrades to the Collections libraries, default methods, and the `Stream` abstraction as a functional container type for data elements.

B.1 Backward compatibility

One of the most important concepts in the Java platform is that of backward compatibility. The guiding philosophy has always been that code that was written or compiled for an earlier version of the platform must continue to keep working with later releases of the platform. This principle allows developers to have a greater degree of confidence that an upgrade of their Java platform software will not impact currently working applications.

As a consequence of backward compatibility, limitations to the ways in which the platform can evolve exist—and these limitations affect developers.

NOTE To be backward compatible, the Java platform may not add additional methods to an existing interface within the JDK.

To see why this is the case, consider the following: if a new version of a certain interface `IFoo` were to add a new method `newWithPlatformReleaseN()` with release N of the

Java platform, all previous implementations of `IFoo` that were compiled with platform version N–1 (or earlier) would be missing this new method, which would cause a failure to link old implementations of `IFoo` under Java platform version N.

This limitation was a serious concern for the JDK 8 implementation of lambda expressions because a primary design goal was to be able to upgrade standard JDK data structures to implement coding idioms from the functional school of programming. The intent was to add new methods that use lambda expressions to express functional ideas (such as `map()` and `filter()`) throughout the Java Collections libraries.

B.2 Default methods

To solve this problem, an entirely new mechanism was needed. The goal was to allow the upgrade of interfaces with new releases of the Java platform by adding *default methods*.

NOTE From Java 8 onward, a default method (sometimes called an optional method) can be added to any interface. This must include an implementation, called the *default implementation*, which is written inline in the interface definition. This change represents an evolution of the interface definition and does not break backward compatibility.

The rules governing default methods follow:

- Any implementation of the interface *may* (but is not required to) implement the default method.
- If an implementing class implements the default method, the implementation in the class is used.
- If an implementing class does *not* implement the default method, the default implementation (from the interface definition) is used.

Let's take a quick look at an example. One of the default methods that was added to `List` in JDK 8 is the `sort()` method. Its definition follows:

```
public default void sort(Comparator<? super E> c) {
   Collections.<E>sort(this, c);
}
```

This means that any `List` object has an instance method `sort()`, which can be used to sort the list in place using a suitable `Comparator`. Any implementation of `List` can pro-

vide its own override of the `sort()` behavior, but if it does not, this default, which falls back to the implementation provided in the `Collections` helper class, will be available.

The default methods mechanism works via class loading. When an implementation of an interface is being loaded, the class file is examined to see whether all of the optional methods are present. If they are, class loading continues normally. If not, the bytecode of the implementation is patched to add in the default implementation of the missing methods.

NOTE Default methods represent a fundamental change in Java's approach to object orientation. From Java 8 onward, interfaces can contain implementation code. Many developers see this as relaxing some of the rules of Java's strict single inheritance.

Developers should understand one detail of how default methods work: the possibility of *default implementation clash*, which has two parts. First, if an implementing class already has a method that has the same name and signature as a new default method, then the pre-existing implementation will always be used in preference to the default implementation.

Second, if a class implements two interfaces that both contain a default method with the same name and signature, the class must implement the method (and it can choose either to delegate to the interface default or to do something else entirely). This raises the possibility that adding a default method to an interface can break client code because if the client code is already implementing another interface that has a default method, the possibility of implementation clash exists. In practice, however, this situation is very rare, and this possibility is deemed a small price to pay for the other benefits that default methods bring.

B.3 Streams

Recall that one of the goals of Project Lambda was to provide the Java language with the ability to easily express techniques from functional programming. For example, this means that Java acquired simple ways to write `map()` and `filter()` idioms.

In the original design sketch for Java 8, these idioms were implemented by adding these methods directly to the classic Java Collections interfaces as additional default methods. However, this approach was unsatisfactory for several reasons.

For one thing, because `map()` and `filter()` are relatively common names, it was felt that the risk of existing implementations was too high—that many of the user-written imple-

mentations of the Collections would have existing methods that would not respect the intended semantics of the new methods.

Instead, a new abstraction, called a `Stream`, was invented. A `Stream` is a container type, which in some ways is analogous to an iterator for the more functional approach to handling collections and aggregate data.

The `Stream` interface is where all of the new *functionally orientated* methods have been placed, such as `map()`, `filter()`, `reduce()`, `forEach()`, and `flatMap()`. The methods on `Stream` make extensive use of functional interface types, such as lambda expressions as arguments.

A `Stream` is best viewed as a sequence of elements that is consumable. That means that after an element has been taken from a `Stream`, it is no longer available, in much the same way as for an `Iterator`.

NOTE Because `Stream` objects are consumable, they should not be reused or stored in temporary variables. Assigning a `Stream` value to a local variable is almost always a code smell.

The original Collections classes, such as `List` and `Set`, have been given a new default method, called `stream()`. This returns a `Stream` object for the collection, in a similar fashion to how `iterator()` was used in code that uses the classic collections.

3.1 Example

This bit of code shows how we can use a `Stream` and a lambda expression to implement a filter idiom:

```
List<String> myStrings = getSomeStrings();
String search = getSearchString();

System.out.println(myStrings.stream()
                    .filter(s -> s.equals(search))
                    .collect(Collectors.toList()));
```

Note that we also need to call `collect()`—this is because `filter()` returns another `Stream`. To get a collection type back, after our filtering operation, we need to do something to actively convert the `Stream` to a `Collection`.

The overall approach looks like this:

```
        stream()  filter()   map()   collect()
Collection -> Stream -> Stream -> Stream -> Collection
```

The idea is for the developer to build up a "pipeline" of operations that need to be applied to the stream. The actual content of the operations will be expressed using a lambda expression for each operation. At the end of the pipeline, the results need to be materialized back into a collection, so the `collect()` method is used.

Let's look at part of the definition of the `Stream` interface (which defines the `map()` and `filter()` methods):

```
public interface Stream<T> extends BaseStream<T, Stream<T>> {
   Stream<T> filter(Predicate<? super T> predicate);

   <R> Stream<R> map(Function<? super T, ? extends R> mapper);

   // ...
}
```

NOTE Don't worry about the scary-looking generics in those definitions. All the "? super" and "? extends" clauses mean is: "Do the right thing when the objects in the stream have subclasses."

These definitions involve two new interfaces: `Predicate` and `Function`. These can both be found in the `java.util.function` package. Both interfaces have only one method, which doesn't have a default. Therefore, we can write a lambda expression for them, which will be automatically converted into an instance of the correct type.

NOTE Remember that conversion to the correct *functional interface* type (via type inference) is always what the Java platform does when it encounters a lambda expression.

Let's look at a code example. Suppose we're modeling otter populations. Some are wild, and some are in wildlife parks. We want to know how many caged otters are looked after by trainee zookeepers. With lambda expressions and streams, this is easy to do, as shown here:

```
Set<Otter> ots = getOtters();
System.out.println(ots.stream()
   .filter(o -> !o.isWild())
   .map(o -> o.getKeeper())
```

```
    .filter(k -> k.isTrainee())
    .collect(Collectors.toList())
    .size());
```

First, we filter the stream so that only captive otters are handled. Then, we perform a `map()` to get a stream of keepers, rather than the stream of otters (note that the type of this stream has changed from `Stream<Otter>` to `Stream<Keeper>`). Then, we filter again, to select only the trainee keepers, and then we materialize this into a concrete collection instance, using the static method `Collectors.toList()`. Finally, we use the familiar `size()` method to return the count from the concrete list.

In this example, we have transformed our otters into the keepers that are responsible for them. We didn't mutate any state of any otter to do so—this is sometimes called being *side-effect free*.

NOTE In Java, the convention is that code inside `map()` and `filter()` expressions should always be side-effect free. However, this "rule" is not enforced by the Java runtime, so be careful. You should always follow this convention in your own code.

If our use case means that we need to mutate some external state, we could use one of two approaches, depending on what we want to achieve. First, if we want to build up aggregate state (e.g., a running total of the ages of otters), we could use a `reduce()`. Alternatively, if we want to perform a more general state transformation (e.g., transferring otters to a new keeper when the old one leaves), a `forEach()` is more appropriate.

Let's examine how we would calculate the otters average age using the `reduce()` method in the next code snippet:

```
var kate = new Keeper();
var bob = new Keeper();
var splash = new Otter();
splash.incAge();
splash.setKeeper(kate);
Set<Otter> ots = Set.of(splash);

double aveAge = ((double) ots.stream()
    .map(o -> o.getAge())
    .reduce(0, (x, y) -> {return x + y;} )) / ots.size();
System.out.println("Average age: "+ aveAge);
```

First of all we map from the otters to their ages. Next we use the `reduce()` method. It takes two arguments: the initial value (often called the *zero*) and a function to apply step by step. In our example, that is just a simple addition because we want to sum the ages of all the otters. Finally, we divide the total age by the number of otters we have.

Notice that the second argument to `reduce()` is a two-argument lambda. The simple way to think about this is that the first of those two arguments is the "running total" of the aggregate operation and the second is effectively the loop variable as we iterate over the collection.

Finally, let's turn to the case where we want to alter state. For this we will use the `forEach()` operation. In our example, we want to model the Keeper Kate going on holiday, so all her otters should be handed over to Bob for now. This is easily accomplished like this:

```
ots.stream()
.filter(o -> !o.isWild())
.filter(o -> o.getKeeper().equals(kate))
.forEach(o -> o.setKeeper(bob));
```

Notice that neither `reduce()` nor `forEach()` uses `collect()`. `reduce()` gathers up state as it runs over the stream, and `forEach()` is simply applying an action to everything on the stream, so, in both cases, there's no need to rematerialize the stream.

B.4 The limits of collections

Java's Collections have served the language extremely well. However, they are based on the idea that all of the elements of the collection exist and are represented somewhere in memory. This means that they are not capable of representing more general data, such as infinite sets.

Consider, for example, the set of all prime numbers. This cannot be modelled as `Set<Integer>` because we don't know what all the prime numbers are, and we certainly don't have enough heap space to represent them all. In early version of Java, this would have been a very difficult problem to solve within the standard collections.

It is possible to construct a view of data that works primarily with iterators and relegates the underlying collections to a supporting role. However, this requires discipline and is not an immediately obvious approach to the Java Collections. In the past, if the developer wanted to use this type of approach, they would typically depend on an external library that provided better support for this functionality.

Fortunately, Java `Stream`s address this use case by introducing the `Stream` interface as an abstraction that is better suited to dealing with more general data structures than basic finite collections. This means that a `Stream` can be thought of as more general than an `Iterator` or a `Collection`.

NOTE A `Stream` does not manage the storage for elements or provide a way to access individual elements directly from the stream.

However, a `Stream` is not really a data structure—instead, it's an abstraction for handling data, although the distinction between the two cases is somewhat subtle.

B.5 Infinite streams

Let's dig a little deeper into the concept of modeling an infinite sequence of numbers. Some consequences follow:

- We can't materialize the whole stream to a collection so methods like `collect()` won't be possible.
- We must operate by pulling the elements out of the stream.
- We need a bit of code that returns the next element as we need it.

This approach also means that the values of expressions are not computed until they are needed.

Up until Java 8, the value of an expression was always computed as soon as it was bound to a variable or passed into a function. This is called *eager evaluation*, and it is, of course, the default behavior for expression evaluation in most mainstream programming languages.

NOTE With version 8, a new programming paradigm was introduced for Java—`Stream` uses *lazy evaluation* wherever possible.

This is an extremely powerful new feature and does take a bit of getting used to. We discuss lazy evaluation in more detail in chapter 15. The aim of lambda expressions in Java is to simplify life for the ordinary programmer, even if that requires extra complexity in the platform.

B.6 Handling primitives

One important aspect of the Stream API that we have glossed over until now is how to handle primitive types. Java's generics do not allow a primitive type to be used as a type

parameter, so we cannot write `Stream<int>`. Fortunately, the `Streams` library comes with some tricks to help us work around this issue. Let's look at an example:

```
double totalAge = ((double) ots.stream()
                    .map(o -> o.getAge())
                    .reduce(0, (x, y) -> {return x + y;} ));

double aveAge = totalAge / ots.size();
System.out.println("Average age: "+ aveAge);
```

This actually uses primitive types over most of the pipeline, so let's unpack this a bit and see how the primitive types are used in code like this.

First off, don't be confused by the cast to `double`. This is just to ensure that Java does a proper average, instead of performing integer division.

The argument to `map()` is a lambda expression that takes in an `Otter` and returns an `int`. If we could write it using Java's generics, the lambda expression would be converted to an object that implements `Function<Otter, int>`. However, because Java's generics will not allow this, we need to encode the fact that the return type is `int` in a different way—by putting it in the name of the type, so, the type that is actually inferred is `ToIntFunction<Otter>`. This type is known as a *primitive specialization* of the function type and is used to avoid boxing and unboxing between `int` and `Integer`, which saves unnecessary generation of objects and allows us to use function types that are specific to the primitive type that's being used.

Let's break down the average calculation a little more. To take the age of each otter, we use this expression:

```
ots.stream().map(o -> o.getAge())
```

Let's look at the definition of the `map()` method that is being called, shown next:

```
IntStream map(ToIntFunction<? super T> mapper);
```

From this we can see that we are using the special function type `ToIntFunction`, and we're also using a specialized form of `Stream` to represent the stream of ints.

After this, we pass to `reduce()`, which is defined as follows:

```
int reduce(int identity, IntBinaryOperator op);
```

This is also a specialized form that also operates purely on ints and takes a two-argument lambda (both arguments are ints) to perform the reduction.

`reduce()` is a collecting operation (and, therefore, eager), so the pipeline is evaluated at that point and returns a single value, which is then cast to a double and turned into the overall average.

If you missed all of this detail about primitives, don't worry—it's one of the good things about type inferencing: most of these differences can be hidden from the developer most of the time.

Let's conclude by talking about a topic that's often misunderstood by developers: the support for parallel operations on streams.

B.7 Parallel operations?

In old versions of Java (7 and earlier), all operations on collections are serial. No matter how large the collection being operated on, only one CPU core will be used to execute the operation. As datasets get larger, this may become hugely wasteful, and one of the possible goals for Project Lambda was to upgrade Java's support for collections to allow efficient use of multicore processors.

NOTE The lazy evaluation approach for streams allows the lambda expression framework to provide support for parallel operations.

The primary assumption in the Stream API is that creating a stream object (whether from a collection or by some other means) should be cheap, but some operations in the pipeline could be expensive. This assumption allows us to characterize a parallel pipeline like this:

```
s.stream()
   .parallel()
   // sequence of stream operations
   .collect( ... );
```

The method `parallel()` converts a serial stream to parallel operations. The intent of this is to allow ordinary developers to rely on `parallel()` as the entry point to transparent

parallelism and places the burden of providing parallel support onto the library writer rather than the end user.

This sounds great in theory, but in practice, implementation and other details end up detracting from the usefulness of the `parallel()` mechanism. Chapter 16 discusses this in more depth.

Due to these limitations, it is strongly recommended that you avoid parallel streams unless you can prove (using the methods of chapter 7) that your application will benefit from adding them. In practice, the authors have seen less than half-a-dozen cases in the wild where parallel streams are actually effective.

Previous chapter
< Appendix A. Selecting your Java

Next chapter
The Well-Grounded Java Developer, Second Edition >